

AirFlow

```
yc compute instance create --name airbyte-node --ssh-key /Users/zhannamakrusina/yandex-cloud/
key_ssh_id.pub --create-boot-disk image-folder-id=standard-images,image-family=ubuntu-2404-
lts,size=50,auto-delete=true --network-interface subnet-name=default-ru-central1-a,nat-ip-
version=ipv4 --memory 16G --cores 4 --core-fraction 50 --zone ru-central1-a --preemptible --
hostname airbyte-node
done (39s)
id: fhm71qh6qnh3bl0kn1u
folder_id: blg8hdofsv85pjmtq38l
created_at: "2025-11-12T14:07:18Z"
name: airbyte-node
zone_id: ru-central1-a
platform_id: standard-v2
resources:
  memory: "17179869184"
  cores: "4"
  core_fraction: "50"
status: RUNNING
metadata_options:
  gce_http_endpoint: ENABLED
  aws_v1_http_endpoint: ENABLED
  gce_http_token: ENABLED
  aws_v1_http_token: DISABLED
boot_disk:
  mode: READ_WRITE
  device_name: fhmh3o8hs747g38grpth
  auto_delete: true
  disk_id: fhmh3o8hs747g38grpth
network_interfaces:
  - index: "0"
    mac_address: d0:0d:70:ea:26:d5
    subnet_id: e9b7lj0aeh088c2g24af
    primary_v4_address:
      address: 10.128.0.28
    one_to_one_nat:
      address: 51.250.3.155
    ip_version: IPV4
serial_port_settings:
  ssh_authorization: OS_LOGIN
gpu_settings: {}
fqdn: airbyte-node.ru-central1.internal
scheduling_policy:
  preemptible: true
network_settings:
  type: STANDARD
placement_policy: {}
hardware_generation:
  legacy_features:
    pci_topology: PCI_TOPOLOGY_V1
```

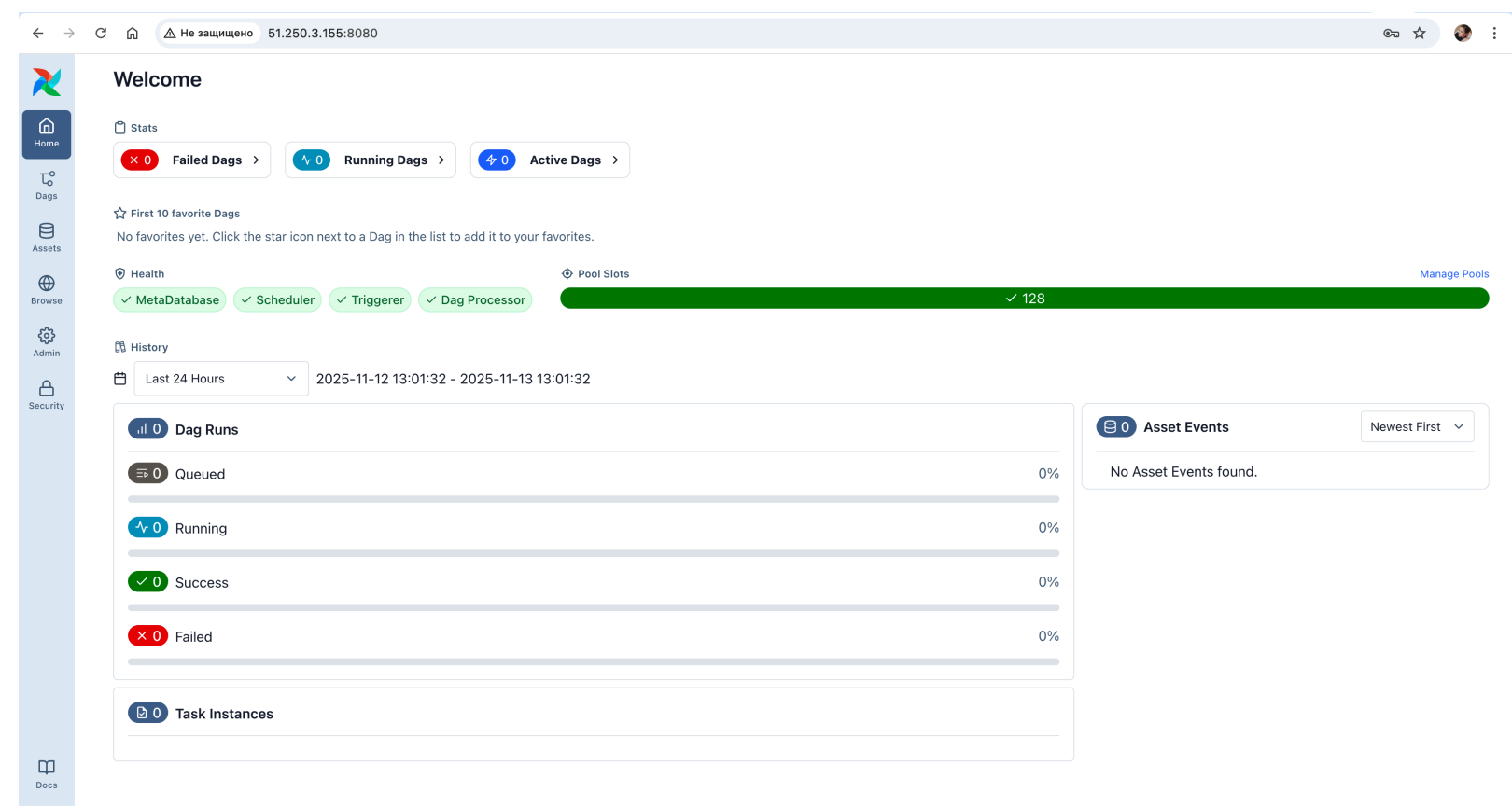
There is a new yc version '0.175.0' available. Current version: '0.154.0'.
See release notes at <https://yandex.cloud/ru/docs/cli/release-notes>
You can install it by running the following command in your shell:
\$ yc components update

По инструкции развернула airflow в docker

<https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html>

```
yc-user@airbyte-node:~$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
feb112d49095   apache/airflow:3.1.2                "/usr/bin/dumb-init ..." 19 hours ago  Up 19 hours  8080/tcp                           yc-user-airflow-worker-1
7c44a6ac971e   apache/airflow:3.1.2                "/usr/bin/dumb-init ..." 19 hours ago  Up 19 hours  8080/tcp                           yc-user-airflow-dag-processor-1
9eb139ab3d83   apache/airflow:3.1.2                "/usr/bin/dumb-init ..." 19 hours ago  Up 19 hours  8080/tcp                           yc-user-airflow-triggerer-1
fdc4994efa65   apache/airflow:3.1.2                "/usr/bin/dumb-init ..." 19 hours ago  Up 19 hours  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  yc-user-airflow-apiserver-1
b5340a0621a1   apache/airflow:3.1.2                "/usr/bin/dumb-init ..." 19 hours ago  Up 19 hours  8080/tcp                           yc-user-airflow-scheduler-1
3dae2ba76b7a   postgres:16                          "docker-entrypoint.s..." 20 hours ago  Up 20 hours  5432/tcp                           yc-user-postgres-1
fccc77172a4c   redis:7.2-bookworm                  "docker-entrypoint.s..." 20 hours ago  Up 20 hours  6379/tcp                           yc-user-redis-1
yc-user@airbyte-node:~$
```

И зашла на веб интерфейс



Теперь создадим переливку данных в ClickHouse

Нужно поставить requests и clickhouse-connect на контейнеры yc-user-airflow-worker-1, yc-user-airflow-dag-processor-1, yc-user-airflow-scheduler-1
sudo docker exec -it <id container> /bin/bash
Pip install clickhouse-connect
Pip install requests
Exit

И перезапуск всех этих трех контейнеров

Затем в папке dags создаем файл jsonplaceholder_to_clickhose_fin.py

```
from datetime import datetime, timedelta
import json
import requests
from airflow import DAG
from airflow.operators.python import PythonOperator
import clickhouse_connect

# Параметры DAG
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
```

```

    'email_on_failure': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=1),
}

def fetch_and_load_to_clickhouse(**context):
    # 1. Запрос к API
    url = "https://jsonplaceholder.typicode.com/posts"
    response = requests.get(url)
    response.raise_for_status()
    data = response.json() # список словарей

    # Проверка: убедимся, что данные есть
    if not data:
        raise ValueError("Пустой ответ от API")

    # 2. Подключение к ClickHouse
    client = clickhouse_connect.get_client(
        host='clickhouse-node', # или 'clickhouse' если в Docker-сети
        port=8123,
        username='default',
        password='', # по умолчанию пустой
        database='default'
    )

    # 3. Преобразуем данные в формат для вставки
    # Ожидается: [(id, userId, title, body), ...]
    rows = [
        (item['id'], item['userId'], item['title'], item['body'])
        for item in data
    ]

    # 4. Вставка в ClickHouse
    client.insert(
        table='posts',
        data=rows,
        column_names=['id', 'userId', 'title', 'body']
    )

    client.close()
    print(f"Успешно вставлено {len(rows)} записей.")

# Определение DAG
with DAG(
    dag_id='jsonplaceholder_to_clickhouse_fin',
    default_args=default_args,
    description='Загрузка постов из JSONPlaceholder в ClickHouse',
    schedule="@daily", # ежедневно
    start_date=datetime(2025, 1, 1),
    catchup=False,
    tags=['etl', 'api', 'clickhouse'],
) as dag:

    extract_and_load_task = PythonOperator(
        task_id='fetch_and_load_posts_fin',
        python_callable=fetch_and_load_to_clickhouse,
    )

```

Перезапускаем контейнеры и находим Dag и запускаем его вручную. До этого создав таблицу posts

```

CREATE TABLE default.posts
(
    `id` UInt32,
    `userId` UInt32,
    `title` String,
    `body` String
)
ENGINE = MergeTree

```

ORDER BY id

http://51.250.75.49:8123

v25.3.5.42, uptime 21 hr

default

password

select * from posts

Run

(Ctrl/Cmd+Enter) ✓ 200 rows in result, 0.00 sec.

100.0%, Read 200 rows, 45.23 KB

#	id	userId	title	body
1	1	1	sunt aut facere repellat provident occaecati excepturi optio reprehenderit	quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto
2	2	1	qui est esse	est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis qui aperiam non debitis possimus qui neque nisi nulla
3	3	1	ea molestias quasi exercitationem repellat qui ipsa sit aut	et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius odio et labore et velit aut
4	4	1	eum et est occaecati	ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur doloremque ipsam iure quis sunt voluptatem rerum illo velit
5	5	1	nesciunt quas odio	repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur dolor neque

Dag

jsonplaceholder_to_clickhose_fin

Dag Run

manual__2025-11-13T11:14:57+00:00

Search Dags

Trigger

Options

00:00:09.000

00:00:04.500

fetch_and_load_posts_fin

✓

✓

manual__2025-11-13T11:14:57+00:00

Success

Add a note

Clear Dag Run

Mark Dag Run as...

Logical Date	Run Type	Start Date	End Date	Duration	Triggering User Name	Dag Version(s)
2025-11-13 14:14:57	manual	2025-11-13 14:15:02	2025-11-13 14:15:12	00:00:09.633	airflow	v1

Task Instances

Asset Events

Audit Log

Code

Details

Search Tasks

All States

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration	
fetch_and_load_posts_fin		Success	2025-11-13 14:15:04	1	PythonOperator	00:00:07.264	

Заголовок